

PRACTICAL : WEEK 11

Task - 1:

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;

    printf("Thread %d started.\n", thread_id);

    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    printf("Thread %d finished.\n", thread_id);

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_id[NUM_THREADS];

    printf("Starting multithreaded counter...\n");
    printf("Number of threads: %d\n", NUM_THREADS);
    printf("Iterations per thread: %d\n\n", ITERATIONS);

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        thread_id[i] = i + 1;

        if (pthread_create(&threads[i],
                          NULL,
                          increment_counter,
                          &thread_id[i]) != 0)
        {
            printf("Error creating thread %d\n", i + 1);
            return 1;
        }
    }
}
```

```

/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("\nAll threads have completed.\n");

printf("Expected counter value: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter value: %d\n",
      counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Counter reached the expected value.\n");
}
else
{
    printf("Race condition detected!\n");
    printf("Some updates were lost due to concurrent access.\n");
}

return 0;
}

```

OUTPUT:

```

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week11
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week11$ nano counter.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week11$ gcc counter.c -o counter -pthread
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week11$ ./counter
Starting multithreaded counter...
Number of threads: 4
Iterations per thread: 100000

Thread 1 started.
Thread 2 started.
Thread 1 finished.
Thread 2 finished.
Thread 3 started.
Thread 3 finished.
Thread 4 started.
Thread 4 finished.

All threads have completed.
Expected counter value: 400000
Actual counter value: 349610
Race condition detected!
Some updates were lost due to concurrent access.

```

OBSERVATION:

The program successfully demonstrated multithreading using POSIX threads. Four threads were created using `pthread_create()`, and each thread concurrently incremented the same shared counter 100,000 times. Since the counter was accessed without synchronization, a race condition occurred, causing some updates to be lost and the actual counter value to differ from the expected value of 400,000 in some executions. `pthread_join()` was used to wait for all threads to complete before displaying the final result.

Task - 2:

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    int thread_id = *(int *)arg;
    printf("Thread %d started.\n", thread_id);

    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }

    printf("Thread %d finished.\n", thread_id);
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_id[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    printf("Starting synchronized counter...\n");
    printf("Number of threads: %d\n", NUM_THREADS);
    printf("Iterations per thread: %d\n\n", ITERATIONS);

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        thread_id[i] = i + 1;

        if (pthread_create(&threads[i],
                        NULL,
                        increment_counter,
                        &thread_id[i]) != 0)
        {
            printf("Error creating thread %d\n", i + 1);
            return 1;
        }
    }
}
```

```

/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("\nAll threads have completed.\n");

printf("Expected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

printf("Actual counter value: %d\n",
        counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Synchronization successful!\n");
    printf("No updates were lost.\n");
}

pthread_mutex_destroy(&lock);

return 0;
}

```

OUTPUT:

```

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week11
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week11$ nano counter_mutex.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week11$ gcc counter_mutex.c -o counter_mutex -pthread
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week11$ ./counter_mutex
Starting synchronized counter...
Number of threads: 4
Iterations per thread: 100000

Thread 1 started.
Thread 2 started.
Thread 3 started.
Thread 4 started.
Thread 1 finished.
Thread 2 finished.
Thread 3 finished.
Thread 4 finished.

All threads have completed.
Expected counter value: 400000
Actual counter value: 400000
Synchronization successful!
No updates were lost.

```

OBSERVATION:

The program successfully demonstrated thread synchronization using a mutex lock. Multiple threads accessed the shared counter, but `pthread_mutex_lock()` and `pthread_mutex_unlock()` ensured that only one thread could modify the counter at a time. As a result, race conditions were prevented and the actual counter value consistently matched the expected value of 400000. The comparison with Task 1 showed that synchronization prevents loss of updates during concurrent execution.

